KGVoyager: Knowledge Graph Agnostic Question Answering via Agentic Navigation

Essam Wisam and Chengkai Li
University of Texas at Arlington
{abdelghanye, cli}@uta.edu

arXiv:2609.01780v2 [cs.IR] 15 Oct 2026

Abstract

Knowledge Graph Question Answering (KGQA) over RDF graphs remains challenging in domain-specific settings, where formal ontologies and curated text–SPARQL pairs are often unavailable. We present KGVoyager, a KG-agnostic agentic architecture that generates SPARQL queries from natural language questions by dynamically discovering graph structure and semantics, requiring only a query endpoint of the underlying graph. Using a think–act–observe loop with search, exploration, and execution tools, KGVoyager maps terms to graph Internationalized Resource Identifiers (IRIs), uncovers structure, and refines queries through execution feedback—all without pre-existing ontologies or examples. Unlike the prior state of the art, KGVoyager requires only a lightweight class index which renders it applicable for far more real-world endpoints. Across four benchmarks, KGVoyager improves F1 by ∼4 points on average compared to the state of the art while cutting cost by ∼38% and runtime by ∼24%.

1 Introduction

RDF knowledge graphs (KGs) provide a robust framework for representing complex, interconnected data across diverse domains [16, 35]. However, utilizing them effectively requires understanding graph-specific data models and technical query languages such as SPARQL [18, 13, 21, 1]. Natural language interfaces that map user questions to SPARQL queries lower this barrier and have driven significant research interest in knowledge graph question answering (KGQA) [9, 14, 22, 42, 32, 19].

LLM-based KGQA systhems have achieved state-of-the-art benchmark performance [9] over earlier machine-learning [6], deep-learning [41, 32, 4], template, and rule-based approaches [13]. These solutions fundamentally leverage ontologies and example text–query pairs for grounding and guiding query generation. Ontologies encode classes, attributes, relationships, and constraints that help models produce semantically valid queries [12, 24, 26]. In addition to ontologies, example text–SPARQL pairs improve performance of such natural language interfaces by serving as in-context examples or facilitating fine-tuning [39, 42, 34]. This dual reliance on ontologies and text–query pairs parallels SQL-based DBQA, where systems depend on the database schema and text–SQL pairs to ground query construction [43].

However, the ontologies and example text–SPARQL pairs on which these systems rely are not always available for domain-specific knowledge graphs. We identify three distinct challenges. (1) *Ontology deficiency*: Unlike relational databases, which require explicit schemas, RDF permits a flexible, schema-optional representation. Many knowledge graphs lack a formal ontological structure. They may define entities, classes, and properties without specifying pivotal semantic relationships such as class hierarchies, property characteristics, and other semantic attributes. (2) *Ontology opacity*: Even when knowledge graphs do incorporate ontological structure, they predominantly lack documentation that exposes or explains such structure and related constraints [15]. (3) *Example scarcity*: Creating text–SPARQL example pairs for new domain-specific knowledge graphs is both costly and challenging, requiring human experts with deep domain knowledge, advanced SPARQL proficiency, and a thorough understanding of the underlying ontology. This is corroborated by the pronounced scarcity of query examples for domain-specific knowledge graphs [30, 27].

These challenges hinder the development of effective KGQA systems for *domain-specific* settings. Most recent natural language to SPARQL systems [22, 42, 32, 19, 4, 10, 34, 39] avoid these constraints by focusing on *open-domain* knowledge graphs such as Wikidata and DBpedia, which

benefit from inherent knowledge in LLMs [14, 39], curated text–SPARQL datasets [11, 36, 37], and community ontology tooling [22, 42]. These assets are rarely available for arbitrary domain-specific knowledge graphs.

As ontologies may be unavailable or opaque and example pairs scarce, we treat a SPARQL query endpoint as the sole universally assumable resource. Our system aims to enable natural language interfaces for arbitrary knowledge graphs *given only query endpoint access*. To achieve this goal, our system must simulate the exploratory learning process a human expert would undertake when encountering an unfamiliar knowledge graph. To this end, we present KGVoyager, an agentic architecture that uses a suite of KG-agnostic tools to dynamically discover latent graph structure, semantics, and instance data. KGVoyager then uses this discovered information to generate SPARQL queries that answer user questions. We design a set of searching, sampling, and navigation tools that enable systematic exploration of arbitrary knowledge graphs. Using these tools within a think-act-observe loop [40], the agent identifies relevant classes and properties, maps local graph structure, and validates query drafts through execution feedback until it derives an accurate SPARQL query.

KGVoyager generalizes across endpoints by relying on two assumptions. First, the knowledge graph follows the W3C RDFS standard for defining types [7], so that class discovery and entity search can be expressed as agentic tools that run unchanged on any such endpoint. Second, a lightweight class index, containing the list of entity types in the graph, can be retrieved from the endpoint with a single SPARQL query; individual entities are then located on demand within the relevant classes. Both assumptions hold broadly in practice: in a scan of knowledge graph endpoints from the Open Knowledge Network (OKN) registry [27], ∼96% of the scanned knowledge graphs follow the RDFS typing standard (Appendix B), and the class index can be built for ∼68% of the tested endpoints under the Apache Jena SPARQL engine and ∼96% under the QLever SPARQL engine (Section 4.1).

To the best of our knowledge, the only prior work sharing a similar endpoint-only, KG-agnostic objective is GRASP [38], which also targets zero-shot SPARQL generation over arbitrary knowledge graphs. Unlike KGVoyager, which requires only an index of the graph's classes, GRASP assumes that an index covering all entities in the graph can be retrieved from the endpoint. This assumption holds for only ∼21% of the tested OKN endpoints under Jena and none under QLever. Other fundamental differences are detailed in Section 2. Even when GRASP is given its full entity index, KGVoyager outperforms it on average across four domain-specific benchmarks, with ∼4 points higher F1, ∼38% lower cost, and ∼24% shorter runtime (Section 4).

KG	KGVoyager (classes)		GRASP (entities)	
	Count	Size	Count	Size
DBLP	35	2.7 KB	140.9M	21 GB
SOCKG	91	5.3 KB	7.1M	1.1 GB

Table 1: Indexed items and on-disk size of KGVoyager's class index vs. GRASP's entity index for the DBLP and SOCKG knowledge graphs.

In summary, our key contributions are:

- **KGVoyager**, an agentic architecture that leverages a suite of KG-agnostic tools to generate SPARQL queries that answer user questions without relying on pre-existing ontologies or curated example pairs.
- **Evaluation** of KGVoyager against the current state-of-the-art across diverse domain-specific knowledge graphs, demonstrating superior performance in correctness, efficiency, and applicability.
- **Open-source release** of KGVoyager, enabling practitioners to deploy the system against any SPARQL endpoint with minimal configuration. The codebase, benchmark queries, results, and demo recordings are available at https://anonymous.4open.science/r/nl2sparql2-C171.

2 Related Work

GRASP [38] is the prior work most similar to KGVoyager, employing an LLM agent with KG-agnostic tools for zero-shot SPARQL generation over arbitrary knowledge graphs. KGVoyager differs in three respects: the scale of required indexing, the inference of property semantics, and the retrieval mechanism. *First*, GRASP indexes every entity, whereas KGVoyager indexes only classes, which scale with the ontology rather than the data. The class index is thus orders of magnitude smaller, as shown for two benchmark knowledge graphs (Table 1). The class index can also be built for far more OKN endpoints (Section 4.1).

Second, GRASP differentiates properties using ontological domain/range annotations, which are not always present (the ontology-deficiency challenge discussed in Section 1); KGVoyager infers them directly from data via dedicated exploration

tools. *Third*, GRASP uses semantic and keyword search as separate signals across its tools, whereas KGVoyager combines them jointly via hybrid retrieval, which has been shown to outperform either signal alone in various applications [20, 31].

ToG [34] pursues KG-agnostic question answering through an agentic LLM that traverses the knowledge graph hop-by-hop. Unlike KGVoyager, which explores latent structure and semantics to construct an executable SPARQL query, ToG traverses instance-level data node-by-node to locate answers. The two systems differ in three respects. *First*, in access assumptions: adapting ToG to a new knowledge graph requires downloading and locally deploying the entire graph and implementing a graph-specific Python traversal interface, which is infeasible in endpoint-only settings; KGVoyager uses a unified tool implementation that works across all knowledge graphs that follow the RDFS typing standard via standard SPARQL. *Second*, in grounding: ToG assumes the presence of an initial set KG entities along with the question from which exploration will begin; KGVoyager explicitly treats class, property, and entity grounding as part of the endpoint-only task. *Third*, in reasoning efficiency: ToG's search process grows proportionally with hop distance, the branching factor of intermediate neighborhoods, and the cardinality of candidate solution nodes, whereas KGVoyager's tool suite relies on SPARQL queries that can express traversals over arbitrarily many hops in a single execution while yielding results of any cardinality. Finally, ToG's evaluation covers only open-domain knowledge graphs, providing limited evidence of generalization to domain-specific knowledge graphs.

Other systems are tailored to a specific knowledge graph or domain. **Spinach** [22] uses Wikidata-specific tools and an in-context learning agent to simulate expert exploration for question answering over Wikidata, while **Expasy** [12] presents a bioinformatics text-to-SPARQL system that mitigates hallucinations via schema validation, but relies on curated text–query pairs and KG-specific schemas. Their tight coupling to specific domains limits generalization to arbitrary knowledge graphs without substantial re-engineering, in contrast to the flexibility of KGVoyager's KG-agnostic architecture.

3 Methodology

KGVoyager is a KG-agnostic agentic reasoning architecture equipped with dynamic exploration tools to discover latent graph structure, semantics, and instances to reflectively generate SPARQL queries from natural language questions. KGVoyager relies on the minimal assumption that the target knowledge graph adheres to W3C RDFS standards for class definitions—a convention adopted across almost all RDF knowledge graphs (Appendix B). Under this assumption, search and exploration tools use KG-agnostic SPARQL queries and run unchanged across KGs.

Given a natural language question, KGVoyager formulates SPARQL generation as an iterative exploration problem. The agent operates in a think-act-observe loop [40], reasoning about what to do next, invoking a tool, and incorporating the result before deciding its next step. Its KG-agnostic tools are organized into three groups: (i) *search tools*, which retrieve candidate classes, node instances, and properties from natural language cues; (ii) *exploration tools*, which inspect how these candidates are used and connected in the knowledge graph; and (iii) an *execution tool*, which runs candidate SPARQL queries and returns feedback. The agent uses search to map question terms to classes, entities, and properties, each identified by an IRI. It then uses exploration to fill gaps in how these elements connect. Finally, it uses execution to validate and refine query drafts until it determines that a SPARQL query correctly answers the question.

Figure 1 summarizes the tool suite. Search tools resolve classes, properties, and entities from natural-language cues. Exploration tools inspect entity neighborhoods and property usage to expose structure not immediately apparent from search results. *execute_query* validates partial or complete query drafts. The complete KGVoyager system prompt and tool specification are included in Appendix G. We prompt the agent to generally follow a search, exploration, and execution sequence, mirroring the natural workflow of a human expert encountering an unfamiliar knowledge graph. Appendix D confirms that models typically follow this ordering. The agent submits the final query through a dedicated *answer* tool to mark task completion. Following GRASP [38], we also allow the agent to terminate early via a *cancel* tool when generating a query is unfeasible for a given question under the knowledge graph.

3.1 Search Tools

Class Search. The *search_all_classes* tool accepts a natural language search term and returns a ranked

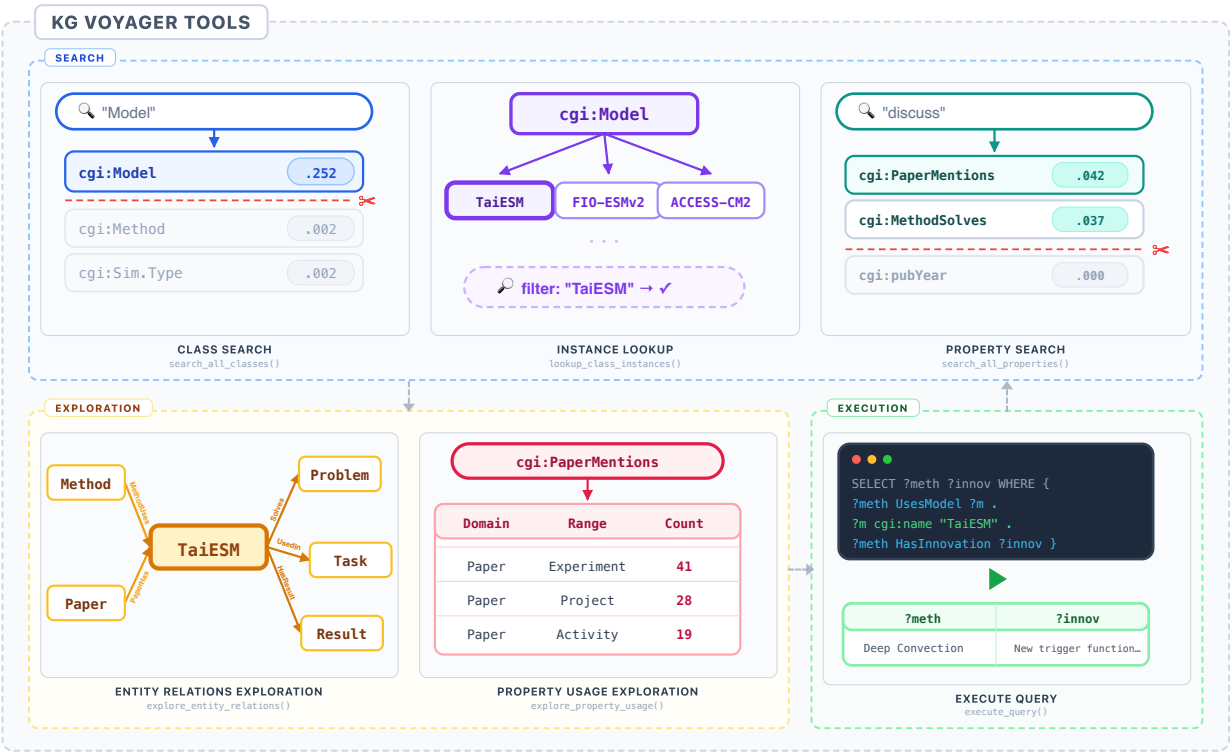

Figure 1: The KGVoyager tool suite. Examples are simplified from runs on the Climate Models knowledge graph.

list of classes whose IRIs or labels are semantically or lexically related to the term. Each result includes the class label, aliases, and instance count. On first invocation, the tool fetches a lightweight class index via SPARQL and caches it locally for all subsequent searches. To retrieve and rank results, we employ a hybrid approach that combines dense vector similarity with sparse BM25 lexical scoring via reciprocal rank fusion [8], which has been shown to consistently outperform either signal alone in various applications [20, 31].

Each result is returned with a relevance score. To prevent low-quality candidates from polluting the agent's context, a phenomenon known as context rot [17], we apply an adaptive filtering mechanism. This mechanism prunes candidates whose relevance score relative to the top-ranked result falls below a ratio δ, while always retaining the top three candidates. We manually tune this hyperparameter and set $\delta = 0.125$.

Instance Lookup. The *lookup_class_instances* tool retrieves instances of a given class, optionally filtered by an *exact_term*. After identifying a candidate class IRI via *search_all_classes*, the agent uses this tool to verify that the class contains the expected content or locate entities mentioned in the question. The default lookup matches the *exact_term* against instance labels and IRIs. Searches using abbreviations, alternative spellings, or overly specific names may return no results because matching relies on lexical SPARQL string filters. To handle such cases, the tool supports a broader *fallback_term* to retrieve a larger candidate set when the *exact_term* search fails. These candidates are then filtered by the semantic similarity of their labels to the *exact_term*. Separately, for knowledge graphs that store useful names or identifiers in literal-valued attributes rather than standard labels, the *lookup_class_instances* tool also supports an *all_attributes* mode that further extends the search to all literal values of class instances, at the cost of slower execution. These class-scoped mechanisms enable precise entity resolution without an exhaustive entity index over the entire graph, which is infeasible for most endpoints.

Property Search. The *search_all_properties* tool searches for properties using an approach similar to that used for class search. Given a search term, it returns a ranked list of property IRIs along with labels, aliases, usage counts as well as domains and ranges, if available. As with class search, the property index is fetched and cached on first use by enumerating all distinct predicates from the endpoint. Subsequent searches apply the hybrid retrieval and adaptive filtering methods used for class search to the locally cached property index.

3.2 Exploration Tools

While the search tools resolve natural language terms to candidate class, property, or instance IRIs, they do not reveal how those IRIs are structurally embedded and connected in the graph. To understand the interaction of properties and the neighborhoods of entities, the agent requires tools that inspect the graph directly.

Property Usage Exploration. Many knowledge graphs lack explicit or accurate domain and range annotations, leading to ambiguity in property interpretation. The *explore_property_usage* tool bridges this gap by examining how a property is actually used. Given a property IRI, it samples triples with that property and aggregates the subject and object classes involved, along with representative instances and their labels. This allows the agent to verify that a candidate property empirically connects the kinds of entities expected by the question, before committing it to the query as well as to disambiguate properties with similar labels. For instance, while a property labeled "located in" connects organizations and companies to cities, a different property "location" may connect events to venues.

Entity Relation Exploration. The tool *explore_entity_relations* inspects the local neighborhood of a given entity. From an entity IRI, it retrieves the incoming and outgoing properties along with example subjects or objects reached through each, including labels when available. This helps understand how the entity is embedded in the graph, particularly when property search has not surfaced the property needed for the question. Given a literal instead, the tool returns the entities with that value and the corresponding data properties.

3.3 Execution Tools

The *execute_query* tool runs a SPARQL query against the endpoint and returns the results. It serves a dual role: providing execution-based feedback from intermediate queries throughout the reasoning process, and executing the final query to obtain the answer. The agent can run intermediate queries to test hypotheses about graph structure, verify that queries produce non-empty results, and diagnose errors in partial queries. This iterative feedback is central to the agent's ability to self-correct and converge on accurate queries.

Figure 2: A complete KGVoyager example. Numbered blocks indicate tool calls in the order they were invoked by the agent. Abbreviations such as 'ds:' denote namespace prefixes used in the underlying graph.

3.4 End-to-End Example

Figure 2 shows KGVoyager resolving the question *"Count the number of papers that Jake Grozdanovski has published in the International Conference on Software and Data Technologies"* in ten tool calls, over the DBLP knowledge graph. During **Search**, the agent decomposes the question into semantic search terms and resolves them to KG elements. The agent infers *author*, implied by "Jake Grozdanovski has published" though not a literal query keyword, and maps it to `ds:Creator` (step 1), then maps *paper* to `ds:Publication` (step 2) and *conference* to `ds:Conference` (step 3). It reuses *author* to surface the authorship property `ds:authoredBy` (step 4), while a search on *"published"* (step 5) returns only journal/book properties, leaving the property linking publications to conference venues unresolved. The agent then grounds the two named entities: the conference name to `dsc:icsoft` (step 6) and the person name to `dp2:9368` (step 7). In the **Explore** phase, the agent inspects `dsc:icsoft` directly, which reveals the incoming relation `ds:publishedInStream`. A targeted follow-up search confirms it is the Publication → Stream property required for the publication-conference relationship. In **Execution**, the agent has fully resolved the triple pattern `?paper ds:authoredBy dp2:9368; ds:publishedInStream dsc:icsoft`, executes the COUNT query, and obtains the answer **1**.

4 Experiments

4.1 Feasibility Analysis

We assessed the feasibility of GRASP's entity index and KGVoyager's class index by attempting to build each over 28 KG SPARQL endpoints from `okn.us` [27], a catalog of graphs developed under the U.S. NSF Proto-OKN initiative. For GRASP, we used its official entity index query. We compared two settings: (i) the individual Apache Jena endpoints of `okn.us` and (ii) its unified federated QLever endpoint [5], which `okn.us` plans to adopt in place of Jena for performance reasons.

Table 2 reports the results. Under Jena, class indexing succeeded for ∼68% of endpoints while entity indexing succeeded for only ∼21%, confirming the impracticality of GRASP's entity-index requirement; the Jena entity-indexing failures returned opaque server errors with no actionable diagnostic information. The gap widens under QLever: class indexing succeeded for ∼96% of endpoints (27/28), while entity indexing failed on all 28, with failures manifesting as server memory crashes—an interpretable outcome given the size of entity indices. These results show that the feasibility of entity-level indexing is highly sensitive to the underlying SPARQL engine and knowledge graph.

Knowledge Graph	Jena Entity	Jena Class	QLever Entity	QLever Class
biobricks-aopwiki	✓	✓	×	✓
biobricks-ice	×	×	×	✓
biobricks-mesh	×	×	×	✓
biobricks-pubchem-annotations	×	✓	×	✓
biobricks-tox21	✓	✓	×	✓
biobricks-toxcast	×	×	×	✓
biohealth	×	✓	×	✓
climatemodelskg	✓	✓	×	✓
dream-kg	✓	✓	×	✓
fio-kg	×	×	×	✓
gene-expression-atlas-okn	×	✓	×	✓
geoconnex	×	✓	×	✓
hydrology-kg	🕓	✓	×	✓
nasa-gesdisc-kg	×	✓	×	✓
nde	×	✓	×	✓
neighborhood-information-kg	✓	✓	×	✓
rural-kg	✓	✓	×	✓
sawgraph	×	×	×	✓
scales	×	✓	×	✓
secure-chain-kg	×	✓	×	✓
semopenalex	×	×	×	✓
sockg	×	✓	×	✓
spatial-kg	🕓	🕓	×	✓
spoke-genelab	×	✓	×	✓
sud-okn	×	×	×	✓
ubergraph	×	✓	×	✓
uf-okn	🕓	✓	×	✓
wikidata	×	×	×	🕓
Success Rate	**21.4%**	**67.9%**	**0%**	**96.4%**

Table 2: Indexing feasibility (✓: Success, ×: Failed, 🕓: Timeout) across 28 KG SPARQL endpoints from `okn.us` under two settings.

4.2 Benchmarks and Models

We evaluated question answering on four domain-specific benchmarks: Climate Models KG [2] (108 questions on climate-science literature), SOCKG [33] (98 questions on soil-carbon modeling), DREAM-KG (88 questions on local government service data), and DBLP-QuAD (100 questions on scholarly metadata). The first three are associated with `okn.us` [27], with question–SPARQL pairs supplied by the respective KG developers. DBLP-QuAD is a public question–SPARQL benchmark; we randomly sampled 100 questions from its test split [3]. For brevity, we refer to each benchmark by the name of its knowledge graph. The question–SPARQL pairs used in our evaluation are available through our repository and in the supplementary data file accompanying this manuscript.

Although Section 4.1 shows that constructing

Benchmark	F1 (%) KGVoyager	GRASP	Δ	Cost ($) KGVoyager	GRASP	Δ	Iterations KGVoyager	GRASP	Δ	Time (s) KGVoyager	GRASP	Δ
Climate Models KG	**69.8**	67.2	+2.6	**2.66**	4.57	−41.8%	**19.6**	23.1	−3.5	**75.1**	119.7	−37.3%
SOCKG	**71.3**	68.9	+2.4	**2.20**	4.00	−45.0%	**19.3**	25.0	−5.7	**57.0**	108.6	−47.5%
DREAM-KG	**49.7**	46.9	+2.8	**3.06**	5.06	−39.5%	**24.2**	30.5	−6.3	**55.9**	92.8	−39.8%
DBLP	**67.5**	60.8	+6.7	**1.79**	2.13	−16.0%	15.1	15.1	0.0	154.2	**128.7**	+19.8%
Average	**64.6**	60.9	+3.7	**2.43**	3.94	−38.3%	**19.6**	23.4	−3.8	**85.6**	112.4	−23.8%

Table 3: Performance by benchmark, averaged across four backbone models. GRASP is evaluated under its best-case setting (entity index available).

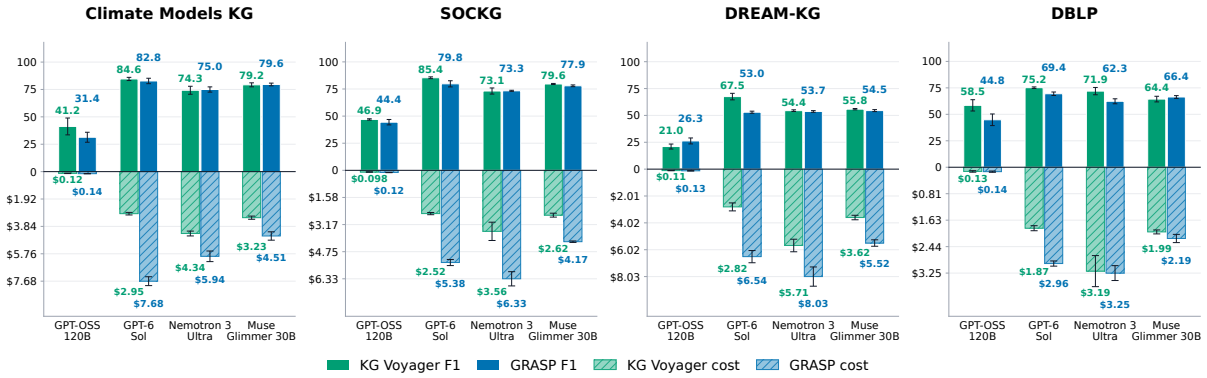

Figure 3: Per-model F1 and monetary cost by benchmark. F1 bars extend above the baseline, cost (hatched) below; error bars indicate run-to-run standard deviation across three independent runs.

GRASP's entity index from endpoint access is infeasible for most knowledge graphs, we evaluate GRASP under its *best-case* conditions by providing the required entity index. We obtained this index through one of three routes: (i) building it directly on relatively small KGs whose Jena endpoint can serve the entity-index query, as for Climate Models KG and DREAM-KG; (ii) locally redeploying the knowledge graph and extracting the index from the deployment, as for SOCKG, whose `okn.us` endpoint fails entity index creation (Section 4.1); or (iii) using a pre-built index, as provided by GRASP's authors for DBLP-QuAD.

KGVoyager assumes only endpoint access. For SOCKG, the KG's authors provided the ontology as a Turtle file, which we use to construct an additional Grep Ontology baseline. This baseline is built on the Unix *grep* command, which coding agents commonly use to search code. It uses the same backbone models with only two tools: *grep* over the ontology file and SPARQL query execution. KGVoyager uses its standard tool suite and is not given the ontology file. This comparison tests whether KGVoyager, using only endpoint access, can outperform a baseline given the full ontology.

We ran our experiments on these benchmarks with four backbone models spanning closed and open systems of varying size and agentic capability. GPT-6 Sol (OpenAI, 2026) is a cost-efficient closed model built for complex coding and agentic workflows [29]. GPT-OSS-120B (OpenAI, 2025) is an open-weight mixture-of-experts reasoning model with 117B total and 5.1B active parameters, built with tool calling support [28]. Nemotron 3 Ultra (NVIDIA, 2026) is an open hybrid Mamba–Transformer mixture-of-experts model with 550B total and 55B active parameters, targeting reasoning and agentic workflows [25]. Muse Glimmer 30B (Meta, 2026) is an open 30B-parameter dense model distilled from Muse Spark for local agentic tasks with failure recovery [23]. The three open models were accessed via OpenRouter. Full implementation hyperparameters are in Appendix A.

4.3 QA Performance

We compared KGVoyager against the prior state of the art, GRASP [38], on the benchmarks and models above. For each combination of method, benchmark and model, we average three independent runs and report F1, monetary cost, agent iterations, and wall-clock time. Following GRASP, we adopt the row-major F1 metric of Spinach [22].

Table 3 reports results averaged across runs and models; Figure 3 shows per-model results. KGVoyager improves F1 on all four benchmarks, with gains ranging from 2.4 percentage points on SOCKG to 6.7 points on DBLP. It also reduces cost on every benchmark and requires fewer agent iterations on all benchmarks except DBLP, where the two systems tie. Runtime is lower by 37–48% on three benchmarks; DBLP is the exception (19.8% higher) because its template-based questions fo-

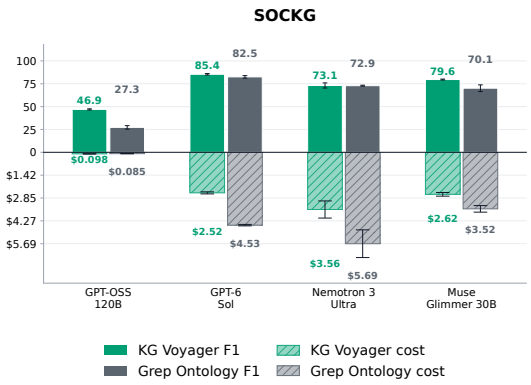

Figure 4: Per-model F1 (top) and monetary cost (hatched, bottom) of KGVoyager and the Grep Ontology baseline on SOCKG; error bars show standard deviation over three runs.

cus heavily on specific entity instances (e.g., paper and author names), which GRASP resolves with a single lookup in its pre-supplied entity index, whereas KGVoyager must first identify the relevant class and then search within it. Overall, KGVoyager also makes fewer tool calls and SPARQL executions per question (Appendix E). We also verify that the benchmarks are nontrivial and analyze performance by query complexity (Appendix C). Finally, we categorize KGVoyager's failure cases in a detailed failure analysis (Appendix F).

Beyond GRASP, we compared KGVoyager against the Grep Ontology baseline on SOCKG (Section 4.2). The results in Figure 4 show that, despite having direct access to the ontology, the baseline trails KGVoyager by 8.1 points of F1 on average (63.2% vs. 71.3%), while KGVoyager also reduces cost by 36.4%. We also observed an 18.7% decrease in runtime. The largest F1 gains are observed for the two smaller models, GPT-OSS-120B (+19.6 points) and Muse Glimmer 30B (+9.5). We attribute these differences to context rot [17], as repeated grep calls return long outputs that accumulate in the context, which raises token cost and tends to degrade smaller models more.

4.4 Ablation Study

We ablate three components of KGVoyager: entity-relation and property-usage exploration, which we remove, and hybrid retrieval, which we replace with keyword-only retrieval in class and property search. Search and execution tools are always retained, as an endpoint-only system cannot ground terms or test queries without them. Each component is evaluated on at least two models and two benchmarks.

Table 4 reports the ablation results. Removing *explore_entity_relations* reduces F1 in all three pairs (by 1.5 to 8.6 points). Removing *explore_property_usage* lowers F1 by 12.5 points on Climate Models KG with GPT-OSS-120B and 4.2 points on SOCKG with Muse Glimmer 30B, while the 0.7-point drop for Nemotron 3 Ultra on SOCKG falls within run-to-run variability. Only GPT-OSS-120B on Climate Models KG pays more cost and iterations with the tool; for the other two pairs, the tool lowers both. Both the full and ablated variants hide the property domain/range search tool, because, per the ontology deficiency challenge (Section 1), we assume such annotations are generally absent, even though some of these KGs may have them. This shows that property-usage exploration recovers useful semantic information directly from graph data when ontological annotations are unavailable. Finally, hybrid retrieval improves F1 over keyword-only search in all four pairs (+1.1 to +26.5 points) and reduces iterations in all four, showing that combining semantic and lexical signals improves search robustness.

	F1 (%)		Cost ($)		Iter.	
Model, benchmark	Full	Abl.	Full	Abl.	Full	Abl.
Entity-relation exploration (w/o *explore_entity_relations*)						
GPT-OSS, SOCKG	**46.9±.6**	38.3±1.1	**.098**	.098	15.9	**15.8**
GPT-6 Sol, SOCKG	**85.4±.8**	83.9±.6	**2.52**	2.74	**10.7**	11.4
GPT-OSS, DBLP	**58.5±5.3**	55.1±1.3	.128	**.081**	**10.8**	12.5
Property-usage exploration (w/o *explore_property_usage*)						
Nemotron 3, SOCKG	**75.5±2.0**	74.8±3.3	**4.46**	5.09	**20.9**	22.4
GPT-OSS, Climate	**33.9±3.8**	21.4±3.7	.116	**.094**	16.6	**14.7**
Muse 30B, SOCKG	**79.9±1.0**	75.7±1.3	**2.72**	2.94	**28.5**	31.3
Hybrid retrieval (keyword-only instead of hybrid)						
GPT-OSS, Climate	**41.2±7.7**	36.3±2.1	.122	**.115**	**16.1**	18.6
GPT-OSS, SOCKG	**46.9±.6**	20.4±2.3	.098	**.094**	**15.9**	19.7
Muse 30B, Climate	**79.2±1.8**	78.1±.9	**3.23**	3.86	**30.7**	33.9
GPT-6 Sol, DREAM	**67.5±3.0**	65.7±1.2	**2.82**	3.13	**12.8**	13.6

Table 4: Ablation study (Abl. = component removed). F1 is mean ± std. over three independent runs; cost and iterations are means.

5 Conclusion

We presented KGVoyager, a KG-agnostic framework that answers natural language questions over arbitrary RDF knowledge graphs given only SPARQL endpoints. Our feasibility study shows that KGVoyager's lightweight class index succeeds on ∼96% of `okn.us` endpoints under QLever, compared to 0% for the entity index required by GRASP. Across four domain-specific benchmarks and four backbone models, KGVoyager improves F1 over GRASP while generally reducing cost, runtime, and iterations, showing that endpoint-only KGQA is viable where curated resources are unavailable.

Limitations

Although KGVoyager requires no KG-specific ontology, text–query examples, or graph-specific tools and assumes that the knowledge graph follows the RDF standard for defining types (`rdf:type`), which is broadly adopted in practice (Appendix B), knowledge graphs that use their own typing conventions, such as Wikidata, are not directly supported. Supporting them would require adapter tools, which we leave for future work.

A second limitation is data availability: domain-specific text–SPARQL benchmarks remain scarce, and despite reaching out to many KG authors and maintainers, we could not gather larger datasets. More public domain-specific KGQA benchmarks are needed and we envision that automated benchmark generation could be promising.

Ethical Considerations

All knowledge graphs endpoints and benchmarks used in this work are publicly available (see links in our repository [1]), and no private or personal data was collected or processed in any part of our experiments. We use existing artifacts only for their intended research and benchmarking purposes. Released artifacts are intended for research, evaluation, and reproducibility, with third-party resources remaining subject to their original terms.

Users of KGVoyager should be aware that, like any LLM-based system, it may generate syntactically valid SPARQL queries that are nonetheless semantically incorrect. In domains where query results inform decisions, such as scientific or policy contexts, outputs should be verified rather than trusted uncritically.

Finally, LLM inference carries an environmental cost. We note, however, that KGVoyager reduces monetary cost by ∼38% and wall-clock runtime by ∼24% relative to the prior state-of-the-art system, partially mitigating this concern.

References

[1] Bilal Abu-Salih. 2021. Domain-specific knowledge graphs: A survey. *Journal of Network and Computer Applications*, 185:103076.

[2] Mustapha Adamu, Qi Zhang, Huitong Pan, Longin Jan Latecki, and Eduard C. Dragut. 2025. Querying climate knowledge: Semantic retrieval for scientific discovery. *Preprint*, arXiv:2509.10087.

[3] Sushil Awale. 2026. DBLP-QuAD: A question answering dataset over the DBLP scholarly knowledge graph. Hugging Face dataset card. `https://huggingface.co/datasets/awalesushil/DBLP-QuAD`.

[4] Debayan Banerjee, Pranav Ajit Nair, Jivat Neet Kaur, Ricardo Usbeck, and Chris Biemann. 2022. Modern baselines for SPARQL semantic parsing. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2260–2265.

[5] Hannah Bast and Björn Buchhold. 2017. QLever: A query engine for efficient SPARQL+Text search. In *Proceedings of the 2017 ACM International Conference on Information and Knowledge Management*, pages 647–656.

[6] Jonathan Berant, Andrew Chlipala, Percy Liang, Bhuwan K Patel, and Luke Zettlemoyer. 2013. Semantic parsing on freebase from question-answer pairs. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1533–1544.

[7] Dan Brickley and R. V. Guha. 2014. RDF Schema 1.1. W3C recommendation, World Wide Web Consortium (W3C).

[8] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759.

[9] Jacopo D'Abramo, Andrea Zugarini, and Paolo Torroni. 2025. Investigating large language models for text-to-SPARQL generation. In *Proceedings of the 4th International Workshop on Knowledge-Augmented Methods for Natural Language Processing*, pages 66–80.

[10] Papa Abdou Karim Karou Diallo, Samuel Reyd, and Amal Zouaq. 2025. FRASE: Structured representations for generalizable SPARQL query generation. *Preprint*, arXiv:2503.22144.

[11] Mohnish Dubey, Debayan Banerjee, Abdelrahman Abdelkawi, and Jens Lehmann. 2019. LC-QuAD 2.0: A large dataset for complex question answering over Wikidata and DBpedia. In *Proceedings of the 18th International Semantic Web Conference*, pages 69–78.

[12] Vincent Emonet, Jerven Bolleman, Severine Duvaud, Tarcisio Mendes de Farias, and Ana Claudia Sima. 2024. LLM-based SPARQL query generation from natural language over federated knowledge graphs. In *Proceedings of the Special Session on Harmonising Generative AI and Semantic Web Technologies (HGAIS 2024) co-located with ISWC 2024*, volume 3953 of *CEUR Workshop Proceedings*, pages 1–10.

[1] `https://anonymous.4open.science/r/nl2sparql2-C171`

[13] Sébastien Ferré. 2013. SQUALL: A controlled natural language as expressive as SPARQL 1.1. In *Natural Language Processing and Information Systems (NLDB)*, volume 7934 of *Lecture Notes in Computer Science*, pages 114–125.

[14] Aleksandr Gashkov, Aleksandr Perevalov, Maria Eltsova, and Andreas Both. 2025. SPARQL query generation with LLMs: Measuring the impact of training data memorization and knowledge injection. In *Proceedings of the 25th International Conference on Web Engineering*, pages 177–192.

[15] Graph.build. 2023. Ontology in graph models and knowledge graphs. Website: https://graph.build/resources/ontology.

[16] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutiérrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Urooj Rashid, Anisa Rula, Lukas Schmelzeisen, Juan F. Sequeda, Steffen Staab, and Antoine Zimmermann. 2021. Knowledge graphs. *ACM Computing Surveys*, 54(4):71:1–71:37.

[17] Kelly Hong, Anton Troynikov, and Jeff Huber. 2025. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma.

[18] Xiangru Jian, Zhengyuan Dong, and M. Tamer Özsu. 2025. InteracSPARQL: An interactive system for SPARQL query refinement using natural language explanations. *Preprint*, arXiv:2511.02002.

[19] Endri Kacupaj, Joan Plepi, Kuldeep Singh, Harsh Thakkar, and Jens Lehmann. 2023. SPARQLGEN: Generating SPARQL queries from natural language questions. In *Proceedings of the ACM Web Conference 2023*, pages 1131–1141.

[20] Saar Kuzi, Mingyang Zhang, Cheng Li, Michael Bendersky, and Marc Najork. 2020. Leveraging semantic and lexical matching to improve the recall of document retrieval systems: A hybrid approach. *Preprint*, arXiv:2010.01195.

[21] Harry Li, Gabriel Appleby, Camelia Daniela Brumar, Remco Chang, and Ashley Suh. 2024. Knowledge graphs in practice: Characterizing their users, challenges, and visualization opportunities. *IEEE Transactions on Visualization and Computer Graphics*, 30(1):584–594.

[22] Shicheng Liu, Sina Semnani, Harold Triedman, Jialiang Xu, Isaac Dan Zhao, and Monica Lam. 2024. SPINACH: SPARQL-based information navigation for challenging real-world questions. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 15977–16001.

[23] Meta Superintelligence Labs. 2026. Muse Glimmer 30B model card. Hugging Face. https://huggingface.co/meta-models/Muse-Glimmer-30B.

[24] Divyansh Shankar Mishra, Abhinav Agarwal, B. P. Swathi, and K. C. 2022. Natural language query formalization to SPARQL for querying knowledge bases using Rasa. *Progress in Artificial Intelligence*.

[25] NVIDIA. 2026. NVIDIA Nemotron 3 Ultra. NVIDIA Research. https://research.nvidia.com/labs/nemotron/Nemotron-3-Ultra/.

[26] Pauline Ochieng, Clifford Kyanda, and Carole Lawrence. 2020. PAROT: Translating natural language to SPARQL. *Expert Systems with Applications*, 150:113281.

[27] OKN.US. 2026. OKN registry: Knowledge graphs registered with the open knowledge network. Website: https://registry.okn.us/registry/.

[28] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b model card. *Preprint*, arXiv:2508.10925.

[29] OpenAI. 2026. Introducing GPT-6 Sol and Luna. OpenAI Blog. https://openai.com/index/introducing-gpt-6-sol-and-luna/.

[30] Julio Cesar Rangel, Tarcisio Mendes de Farias, Ana Claudia Sima, and Norio Kobayashi. 2024. SPARQL generation: an analysis on fine-tuning OpenLLaMA for question answering over a life science knowledge graph. In *Proceedings of the 15th International Conference on Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2024)*, volume 3890 of *CEUR Workshop Proceedings*, pages 36–45.

[31] Jhon Rayo, Raul de la Rosa, and Mario Garrido. 2025. A hybrid approach to information retrieval and answer generation for regulatory texts. In *Proceedings of the 1st Regulatory NLP Workshop (RegNLP 2025)*, pages 31–35.

[32] Md Rashad Al Hasan Rony, Uttam Kumar, Roman Teucher, Liubov Kovriguina, and Jens Lehmann. 2022. SGPT: A generative approach for SPARQL query generation from natural language questions. *IEEE Access*, 10:70712–70723.

[33] Nasim Shirvani-Mahdavi, Devin Wingfield, Juan Guajardo Gutierrez, Mai Tran, Zhengyuan Zhu, and Zeyu Zhang. 2025. A knowledge graph informing soil carbon modeling. In *Proceedings of the 25th International Conference on Web Engineering*, pages 226–241.

[34] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *Proceedings of the Twelfth International Conference on Learning Representations*.

[35] Houcemeddine Turki, Abraham Toluwase Owodunni, Mohamed Ali Hadj Taieb, René Fabrice Bile, and Mohamed Ben Aouicha. 2024. A decade of scholarly research on open knowledge graphs.

In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 438–448, Torino, Italia. ELRA and ICCL.

[36] Ricardo Usbeck, Ria Hari Gusmita, Axel-Cyrille Ngonga Ngomo, and Muhammad Saleem. 2018. 9th challenge on question answering over linked data (QALD-9) (invited paper). In *Joint Proceedings of the 4th Workshop on Semantic Deep Learning (SemDeep-4) and NLIWoD4: Natural Language Interfaces for the Web of Data (NLIWOD-4) and 9th Question Answering over Linked Data Challenge (QALD-9) co-located with 17th International Semantic Web Conference (ISWC 2018)*, volume 2241 of *CEUR Workshop Proceedings*, pages 58–64. CEUR-WS.org.

[37] Ricardo Usbeck, Xi Yan, Aleksandr Perevalov, Longquan Jiang, Julius Schulz, Angelie Kraft, Cedric Möller, Junbo Huang, Jan Reineke, Axel-Cyrille Ngonga Ngomo, Muhammad Saleem, and Andreas Both. 2024. QALD-10 - the 10th challenge on question answering over linked data: Shifting from DBpedia to Wikidata as a KG for KGQA. *Semantic Web*, 15(6):2193–2207.

[38] Sebastian Walter and Hannah Bast. 2025. GRASP: Generic reasoning and SPARQL generation across knowledge graphs. In *Proceedings of the 24th International Semantic Web Conference*, volume 16140 of *Lecture Notes in Computer Science*, pages 271–289. Springer.

[39] Silei Xu, Shicheng Liu, Theo Culhane, Elizaveta Pertseva, Meng-Hsi Wu, Sina Semnani, and Monica Lam. 2023. Fine-tuned LLMs know more, hallucinate less with few-shot sequence-to-sequence semantic parsing over Wikidata. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5778–5791.

[40] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations*.

[41] Xiaoyu Yin, Dagmar Gromann, and Sebastian Rudolph. 2021. Neural machine translating from natural language to SPARQL. *Future Generation Computer Systems*, 117:510–519.

[42] Hamada Mohamed Abdelsamee Zahera, Ali Manzoor, Mohamed Sherif, Diego Moussallem, and Axel-Cyrille Ngonga Ngomo. 2024. Generating SPARQL from natural language using chain-of-thoughts prompting. In *Proceedings of the 20th International Conference on Semantic Systems*, Studies on the Semantic Web, pages 353–368. IOS Press.

[43] Xiaohu Zhu, Qian Li, Lizhen Cui, and Yongkang Liu. 2024. Large language model enhanced text-to-SQL generation: A survey. *Preprint*, arXiv:2410.06011.

Appendix

The appendix is organized as follows. Appendix A lists implementation details and hyperparameters. Appendix B measures how widely `rdf:type` is used across `okn.us` endpoints, supporting KGVoyager's RDF typing assumption. Appendix C analyzes performance as a function of query complexity. Appendix D examines when the agent uses each tool group, and Appendix E reports per-question tool-call and SPARQL-execution counts. Appendix F presents a failure analysis of KGVoyager, and Appendix G gives the full system prompt and tool specification.

A Implementation Details

Table 5 lists the key implementation parameters used across all experiments.

Parameter	Value
Embedding model	`BAAI/bge-small-en-v1.5`
Lexical retrieval	BM25 top-k: 20
Dense retrieval	Dense top-k: 10
Fusion	RRF, equal weights, rank constant 61
Re-ranker	`BAAI/bge-reranker-base`, top-k: 10
Adaptive filtering δ	0.125, minimum 3 rows retained
Property-usage sample size	3000 triples
Vector index	In-memory, cosine similarity
Caching	Embeddings stored on disk; reused unless metadata changes

Table 5: Implementation parameters.

B Prevalence of RDFS Typing

KGVoyager makes only one structural assumption: instances are assigned to classes via the W3C RDFS predicate `rdf:type` [7]. Its class index and tools use this predicate to discover classes. To check how practical this assumption is, we counted typing triples and all triples via a SPARQL query on the 28 `okn.us` knowledge graphs from our feasibility study (Section 4.1). The `wikidata` endpoint returned an error page instead of query results; however, Wikidata is known not to use `rdf:type` for typing entities, as it relies on its own type system.

We consider a graph to follow the RDFS typing standard if it contains at least 1,000 typing triples. Of the 28 graphs, 27 (∼96%) meet this criterion (median of ∼7.1 million typing triples over the 27 graphs that returned a count; Figure 5), suggesting that the assumption is reasonable for most knowledge graphs. The only exception is `wikidata`. This scan is unlikely to be biased by platform policy, since `okn.us` only asks contributors for registry

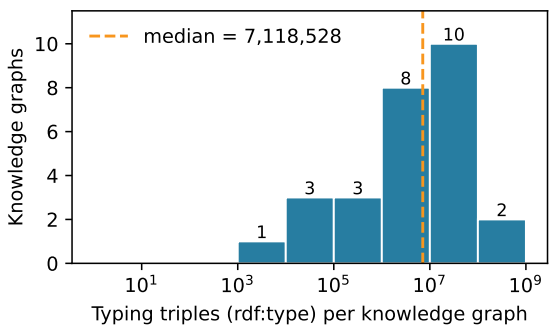

Figure 5: Number of `rdf:type` triples per `okn.us` knowledge graph (27 graphs that returned a count; bins are powers of ten).

metadata and does not prescribe how entities are typed [27]. The prevalence of `rdf:type` instead reflects its status as the W3C-recommended standard for assigning resources to classes [7].

C Query Complexity Analysis

To analyze performance as a function of question complexity and to ensure the benchmarks used for evaluation are nontrivial, we parsed the gold SPARQL queries corresponding to the benchmark questions and measured the complexity of the underlying query subgraph by counting triple patterns in the translated SPARQL algebra, together with the number of distinct special SPARQL constructs (aggregation, `FILTER`, `OPTIONAL`, `UNION`, `MINUS`, `EXISTS`, subqueries, and property paths). We computed paired F1 differences (KGVoyager minus GRASP) for each question, averaging the three runs within each backbone model and then averaging across models.

The benchmarks differ in the complexity of their gold queries, measured by the number of triple patterns, which reflects the size of the subgraph matched by the SPARQL query, and by the number of special SPARQL constructs, as shown in Figure 6. DBLP contains simpler queries than other benchmarks, with at most four triple patterns, whereas the other benchmarks mostly contain queries with five or more. We observe that KGVoyager's improvement over GRASP drops for the most complex queries, on which both systems perform worse; for example, pooled F1 is 58.8% for KGVoyager and 57.4% for GRASP on queries with seven or more triple patterns, compared with 66.7–70.8% and 62.9–67.4% on queries with three to six.

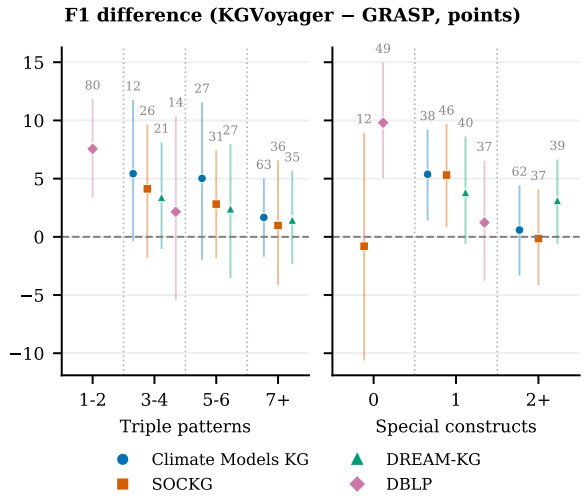

Figure 6: Mean paired F1 difference (KGVoyager minus GRASP) per benchmark by number of triple patterns (left) and distinct special SPARQL constructs (right) in the gold query. Points show means with 95% bootstrap confidence intervals over questions; numbers above the intervals are the number of questions; bins with fewer than 10 questions are omitted.

D Tool-Use Dynamics

We analyze whether the agent follows the search–explore–execute progression described in Section 3. For each tool call, we compute its relative step index, defined as position divided by total run iterations, and pool values across the three KGVoyager runs per benchmark-model pair.

Figure 7 shows all four models. All follow the intended temporal structure on average: search early, exploration in the middle, and execution later. GPT-6 Sol, the best-performing model (78.2% F1), shows the cleanest phase separation, while GPT-OSS-120B, the weakest (41.9% F1), shows the most overlap. The other two models have similar F1 (68.4% and 69.8% for Nemotron 3 Ultra and Muse Glimmer 30B) and similar overlap (0.34 and 0.33). This shows that phase mixing can provide some signal about model performance.

E Tool Invocation Distributions

Figure 8 reports the distribution of tool invocations per question (top) and the number of explicit SPARQL executions per question (bottom). Both analyses pool agent executions across all questions in the four benchmarks, four backbone models, and three runs. The distributions show that KGVoyager requires fewer tool invocations (19.2 vs. 22.2 on average) and SPARQL executions (7.5 vs. 11.4) per question than GRASP, which indicates a more efficient interaction with the knowledge graph.

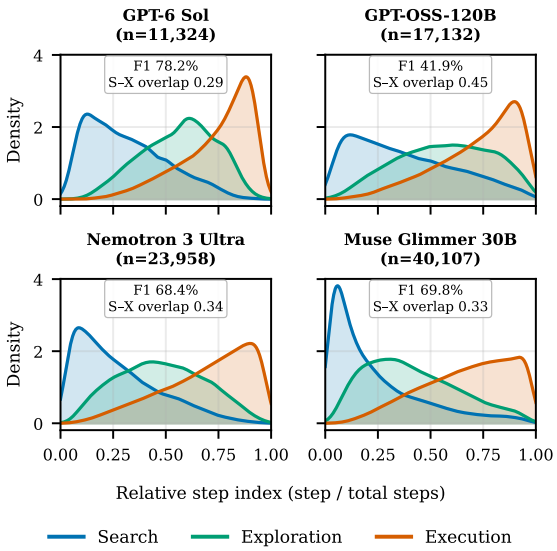

Figure 7: Relative timing of tool use by model: kernel density estimates of the relative step index for the search, exploration, and execution tool groups. Each panel reports the model's KGVoyager F1 averaged over the four benchmarks and the S–X overlap, the overlap coefficient between the search and execution densities (lower means cleaner phase separation).

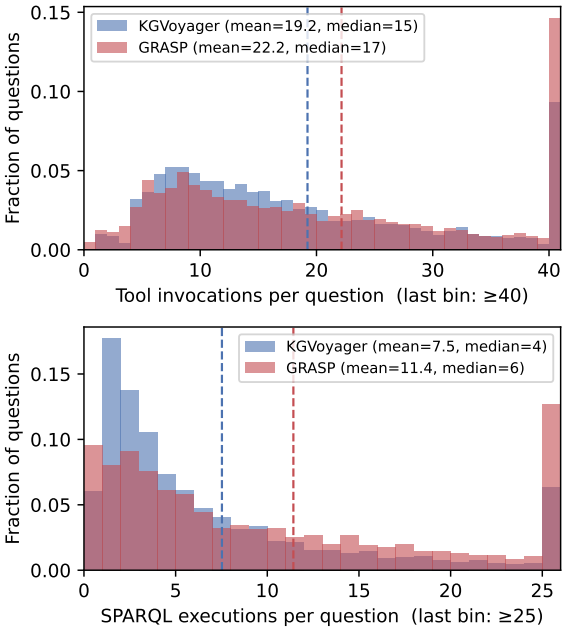

Figure 8: Distribution of tool invocations (top) and explicit SPARQL executions (bottom) per question. Dashed lines indicate means.

F Failure Analysis

To understand the scenarios in which KGVoyager breaks down, we analyze complete failures: (model, KG, question) triples that scored an F1 of 0 across all three independent runs. Considering all the valid gold queries, this criterion yields 156 complete failures across the four models and benchmarks. We attribute each failure to a primary cause using a two-level taxonomy of five coarse-grained error groups, each split into fine-grained categories.

Table 6 summarizes the coarse-grained distribution. Tables 7–11 detail the fine-grained categories with representative examples.

The largest group, query generation/execution (36%), consists almost entirely of early stopping, as seen in Table 10, mostly by smaller models such as GPT-OSS-120B that appear to struggle with long contexts. Another 31% represent genuine reasoning errors (concept resolution and query construction). As seen in Tables 7 and 8, these are spread evenly across wrong classes, properties, and entities, and across missing constraints and incorrect joins. Answer representation (18%; Table 9) covers answers whose content is analogous to that of the gold query but whose form or shape differs. For example, returning paper IDs instead of paper names or using a SELECT query instead of an ASK query for a yes/no question.

Group	Definition	n	%
Concept Resolution	Agent linked a question concept to the wrong instance, class, or predicate IRI	29	19
Query Construction	Correct terms, but wrong constraints, relation direction, or join path	20	13
Answer Representation	Semantically analogous answer scored wrong due to not matching result form or shape	28	18
Query Gen. / Execution	No usable final query was produced or a well-formed query failed to execute	56	36
Other	Cases not fitting the above	23	15

Table 6: Coarse-grained error distribution over 156 LLM-classified complete failures (primary label per case).

Category	n	Summarized example
Wrong Class	11	used c:City instead of c:No_Country_Region
Wrong Property	9	used dblp:publishedInStream instead of dblp:publishedIn
Wrong Entity	9	used pid_94:5174 instead of pid_04:1616 (a different author)

Table 7: Concept resolution errors (29 cases).

Category	n	Summarized example
Missing Constraint	10	omitted required COUNT(DISTINCT ?oceanTopic)
Incorrect Join	10	used service → schema:audienceType instead of service → schema:category → audience → schema:audienceType

Table 8: Query construction errors (20 cases).

Category	n	Summarized example
Result Shape	20	used ?paper ?paperName ?metricCount instead of ?paperId ?metricCount
ASK vs. SELECT	4	used SELECT ?paper ?pid instead of ASK
IRI vs. Label	4	used "Ammonium Nitrate 34-0-0" instead of individuals:AmendmentType.30

Table 9: Answer representation errors (28 cases).

Category	n	Summarized example
Agent Gave Up	50	Agent gave up after being mostly stuck in the search phase for 20 iterations
No Valid Query	6	The submitted query used `rdfs:label` without declaring the `rdfs:` prefix

Table 10: Query generation/execution errors (56 cases).

Category	n	Summarized example
Ambiguous Question	10	not clear how to rank the top 5 papers: all 4,482 papers tie at one distinct method
Top-k / LIMIT Tie-Break	3	used ORDER BY DESC(?methodCount) ?name instead of ORDER BY DESC(?methodCount)

Table 11: Other errors (23 cases). The remaining 10 cases fall into rare miscellaneous categories (one or two cases each) that are not shown.

G KGVoyager System Prompt and Tool Specification

You are a SPARQL query generation assistant. Your job is to generate an accurate SPARQL query that answers the user's question by dynamically discovering the knowledge graph's structure.
You operate in a think-act-observe loop: reason about what you need, invoke a tool, then interpret the results before deciding the next action. Follow this workflow:

1. **SEARCH:** Identify the classes, properties, and entities implied by the question. Search for candidate class and property URIs using natural-language terms. If the question mentions named entities, resolve them by first finding their class, then looking up instances by name. Use relevance scores and usage counts to select the most likely candidates.
2. **EXPLORE:** If the search results do not clearly reveal how properties connect entity types, explore property usage patterns to see actual domain/range types and example triples. If you need to understand the neighborhood of a specific entity, explore its relations.
3. **CONSTRUCT AND TEST:** Build the SPARQL query incrementally using the discovered URIs. Execute intermediate drafts to verify correctness. If results are unexpected, revisit earlier steps – you may have the wrong property, class, or triple pattern. Once the query returns the expected results, call the answer function. If you cannot find a satisfactory query, call cancel.

Keep in mind:

- Always strategically decide which tool call to make next. Include reasoning about your objective with the tool call whenever possible
- Never make a redundant tool call. You will be highly penalized if you repeat a tool call or perform tool calls of very little value (eg, verifying something already obvious). It's absolutely normal if you only use a subset of the available tools to solve the problem.
- Overall solving this with the least amount of tool calls but the perfect accuracy is the ultimate goal.
- Never return a SPARQL query before verifying it is correct and returns results that answer the question.
- For questions with a "True" or "False" answer the SPARQL query should be an ASK query.

Available knowledge graphs:
`{{ knowledge_graphs }}`

Figure 9: KGVoyager system prompt.

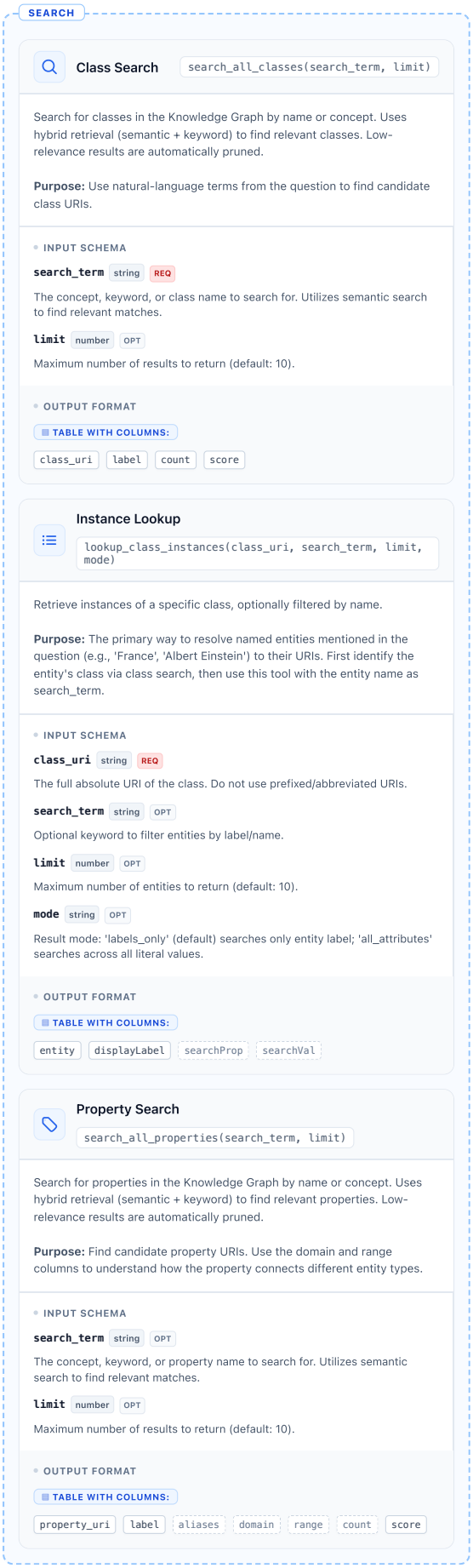

Figure 10: KGVoyager tool specification: search tools.

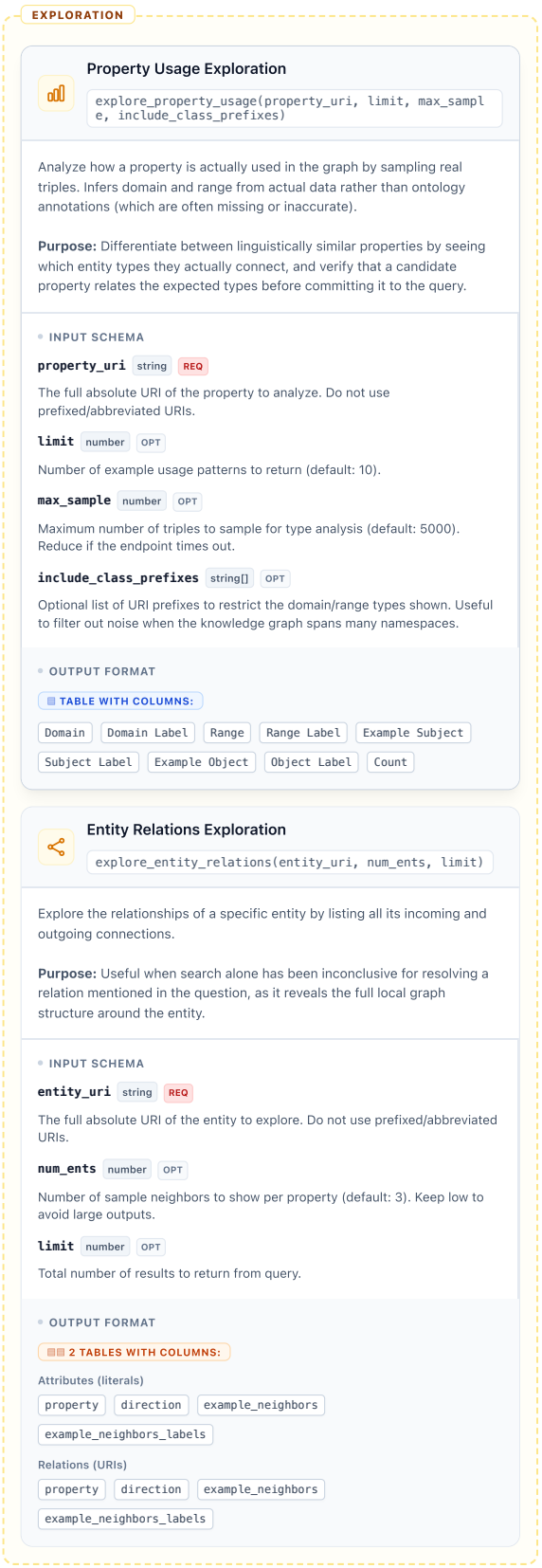

Figure 11: KGVoyager tool specification: exploration tools.

Figure 12: KGVoyager tool specification: execution tools.